

Desempeño y Optimización para la Latencia

Desempeño y Optimización para la Latencia

El desempeño y optimización para la latencia en sistemas distribuidos se refiere a las estrategias y técnicas utilizadas para mejorar la eficiencia y reducir los retrasos en:

- La transmisión de datos
- Procesamiento dentro de estos sistemas.

Desempeño o Performace.

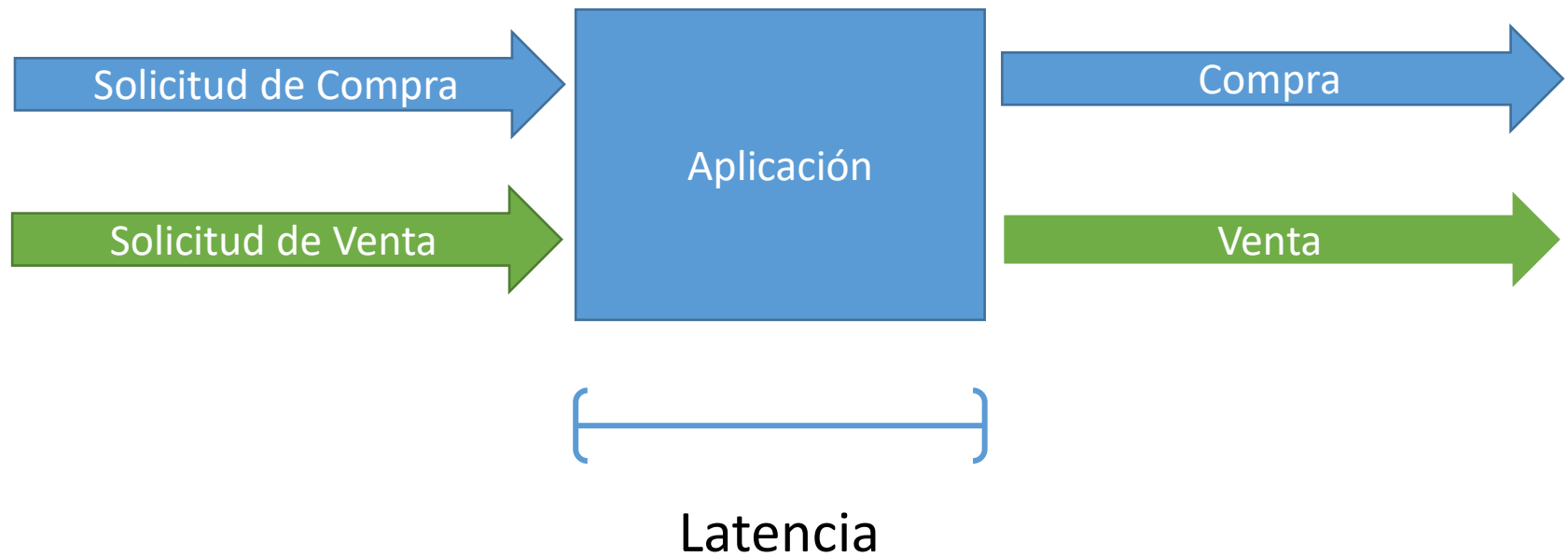
Desempeño se refiere a qué tan eficiente o efectiva es una operación, sistema o tecnología al realizar sus funciones. Este término generalmente abarca aspectos como:

- **Velocidad de procesamiento:** Qué tan rápido puede completarse una tarea o proceso.
- **Uso de recursos:** La cantidad de recursos utilizados (como CPU, memoria, energía) para realizar una operación.
- **Confiabilidad:** Consistencia en la entrega de resultados sin fallos.
- **Escalabilidad:** La capacidad del sistema de manejar un aumento en la carga o demanda sin degradar su desempeño.

Latencia

- Es el tiempo que tarda una acción en generar una respuesta, y puede ser afectada por factores como la distancia física, la congestión de la red, las limitaciones de hardware y las ineficiencias de software

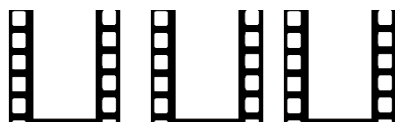
Sistema de Comercio de Alta Velocidad



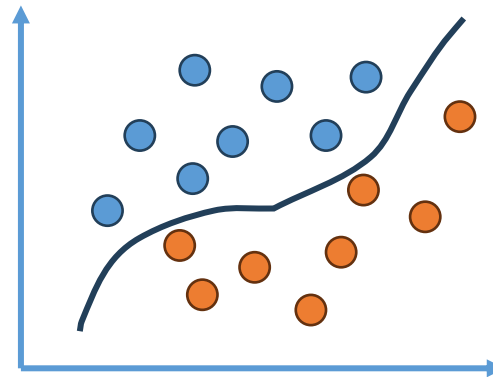
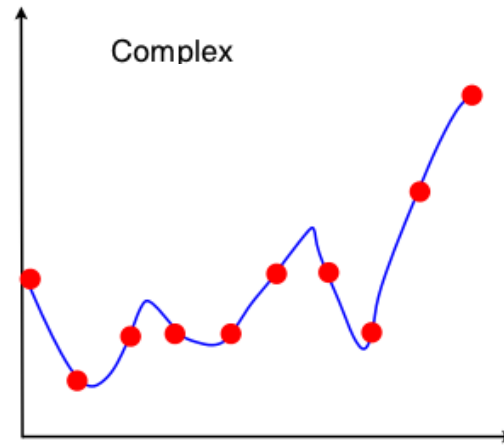
Optimizar la latencia

- Implica identificar y abordar cuellos de botella en componentes como el retraso de propagación, el retraso de transmisión, el retraso de procesamiento y el retraso en las colas de espera.
- Esto puede lograrse mediante el uso de algoritmos eficientes, hardware actualizado y estrategias de diseño que minimicen la distancia y la congestión en la red.

Reproductor de Video



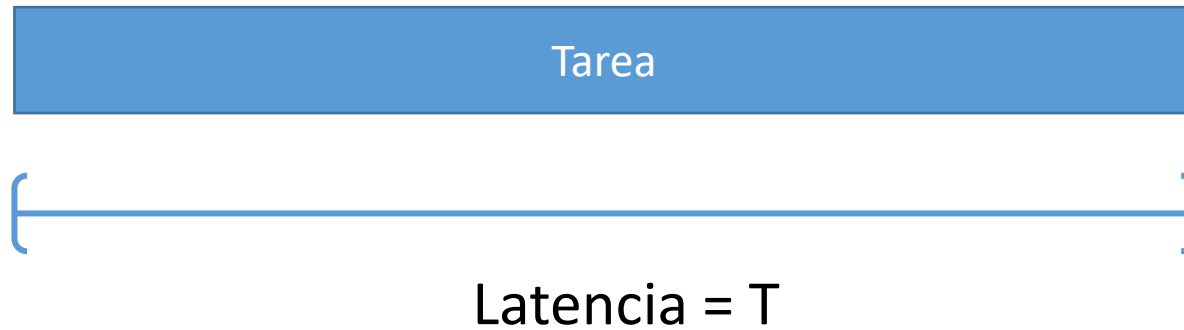
Machine Learning



Rendimiento en subprocesos múltiples

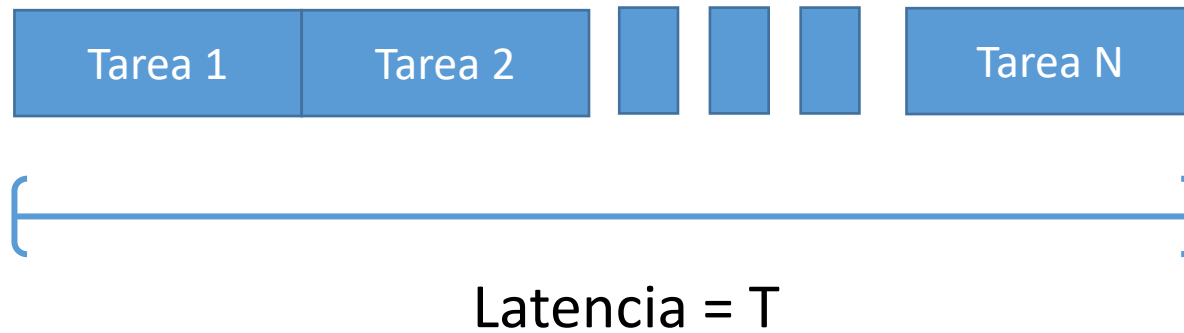
- Latencia: El tiempo para completar una tarea. Medido en **unidades de tiempo**
- Throughput (Rendimiento): la cantidad de tareas completadas en un período determinado. Medido en **unidades de tareas/tiempo**

Latency (latencia)



T: una tarea

Latency (latencia)



T: una tarea

Ti: múltiples tareas independientes

Latency (latencia)



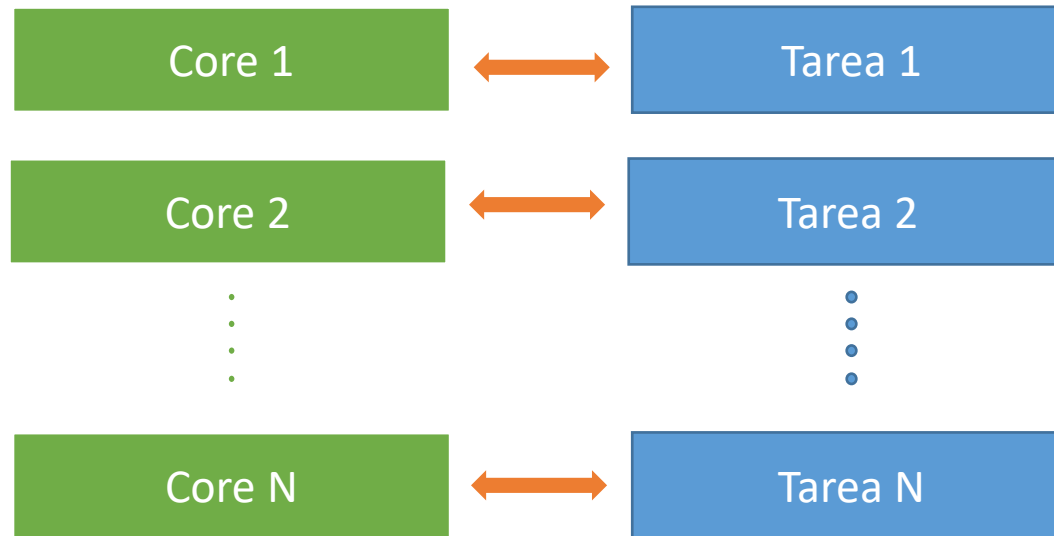
N: numero de sub tareas

Latency (latencia)

- Es un enfoque teórico para reducir la latencia en un factor N , y mejorar el rendimiento en un factor de N .
- $N=?$
 - ¿Cuántas subtareas/hilos podemos tener, para poder dividir la tarea original?
- ¿Podemos dividir cualquier tarea en subtareas?

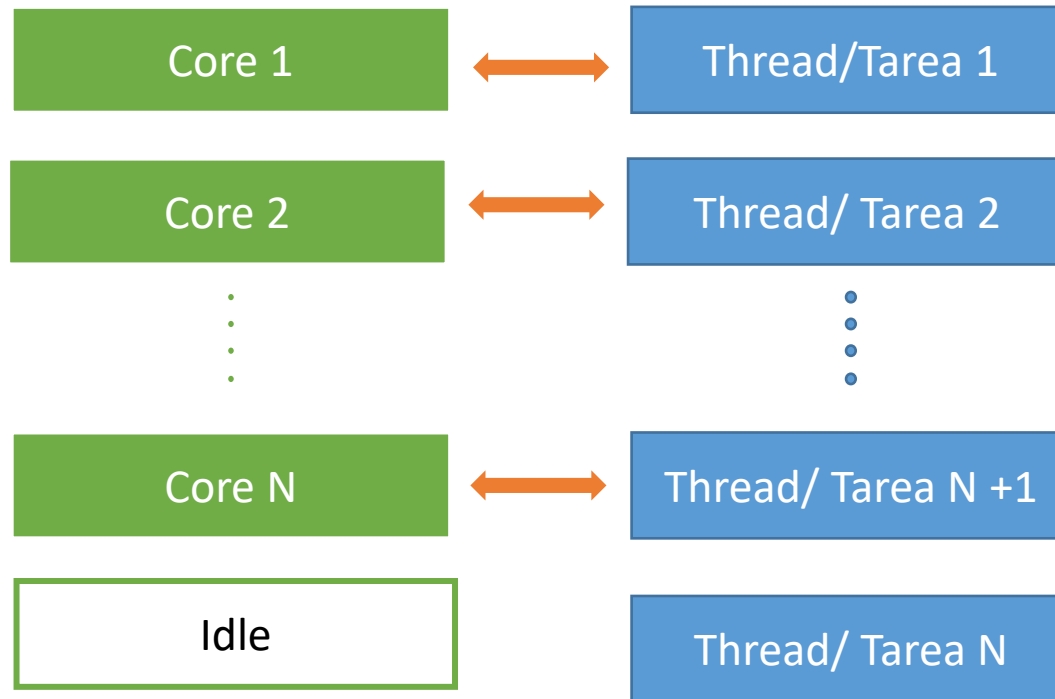
$N=?$

- En una computadora de propósito general: N = numero de núcleos



N=?

- Con un hilo extra y tareas = $N + 1$



N=?

- #threads = # cores es óptimo solo si todos los subprocesos son ejecutables y pueden ejecutarse sin interrupción (sin I/O/bloqueo de llamadas/sleep, etc.)
- La suposición es que no se está ejecutando nada más que consuma una gran cantidad de CPU

¿Cuál es la diferencia entre Hyper Threading y la tecnología Multi-Core?

Hyper-Threading Technology

- La tecnología Hyper-Threading crea dos núcleos de procesamiento virtual para cada núcleo físico presente en una CPU. El **physical core (núcleo físico)** alimenta los **virtual cores (núcleos virtuales)**, que luego comparten la responsabilidad del procesamiento de tareas. Cada núcleo virtual es idéntico al otro y, aunque ninguno es tan potente como el núcleo físico, juntos superan con creces la potencia del núcleo físico

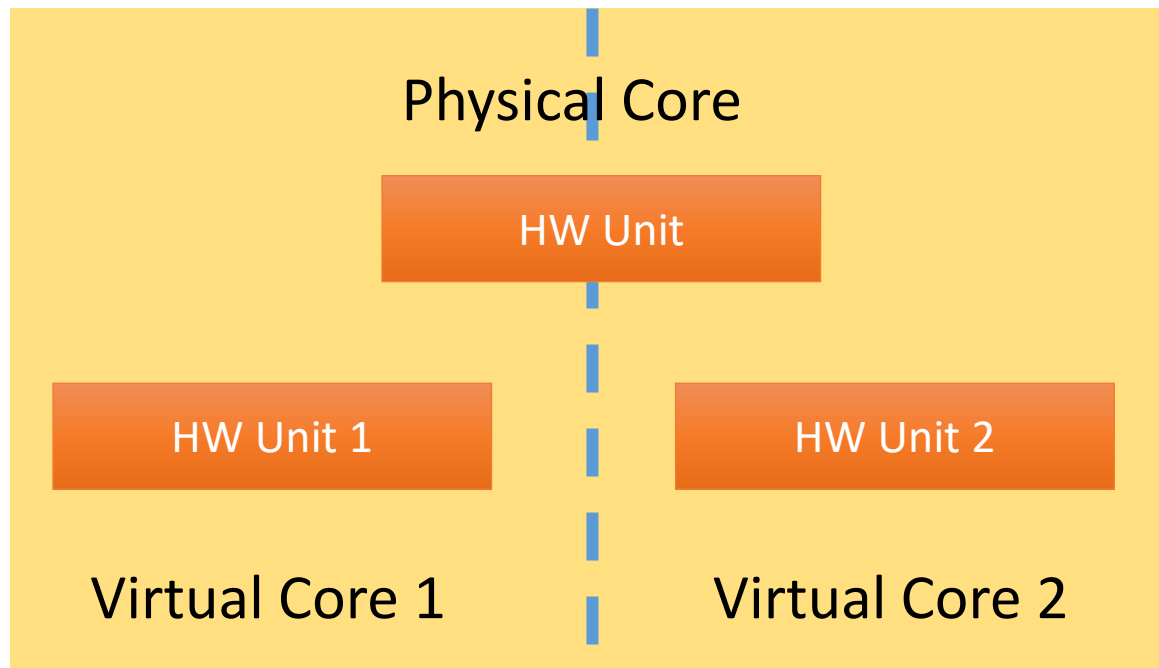
¿Cuál es la diferencia entre Hyper Threading y la tecnología Multi-Core?

Multi-Core Technology

- La tecnología multinúcleo, que suele estar disponible en CPU de dos núcleos, cuatro núcleos y hexa-núcleos, es una tecnología que agrega núcleos de procesamiento físico adicionales. En una CPU de un solo núcleo, las tareas se procesan una a la vez por orden de llegada. Esto puede ser problemático para la multitarea, ya que las tareas comenzarán a respaldarse. En un procesador con dos o más núcleos, la multitarea es mucho más eficiente ya que hay múltiples núcleos disponibles para procesar tareas. Cuantos más núcleos tenga, más datos podrá procesar sin que disminuya el rendimiento.

N=?

- Hyper Threading – Virtual Cores vs Physical Cores

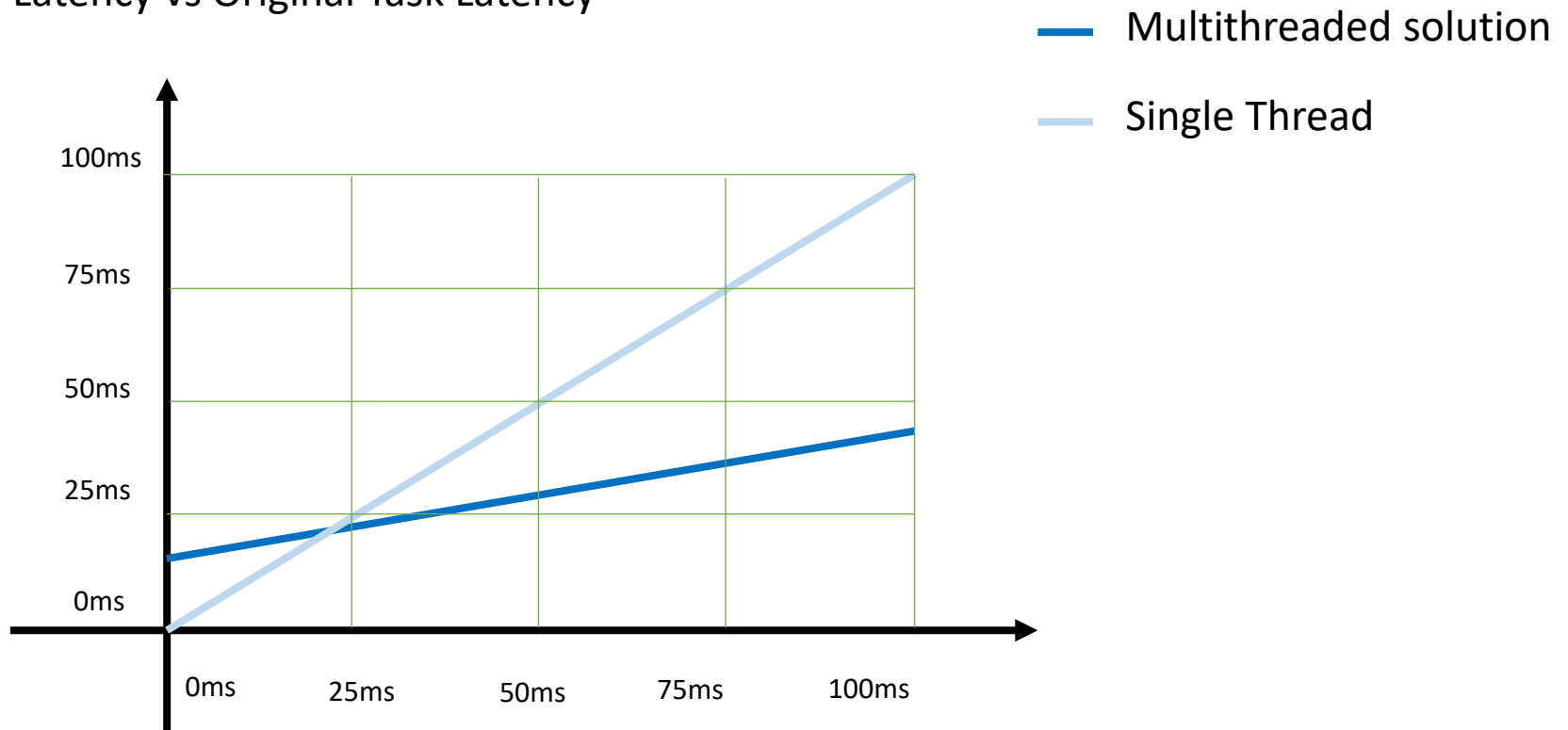


Costo inherente de paralelización y agregación

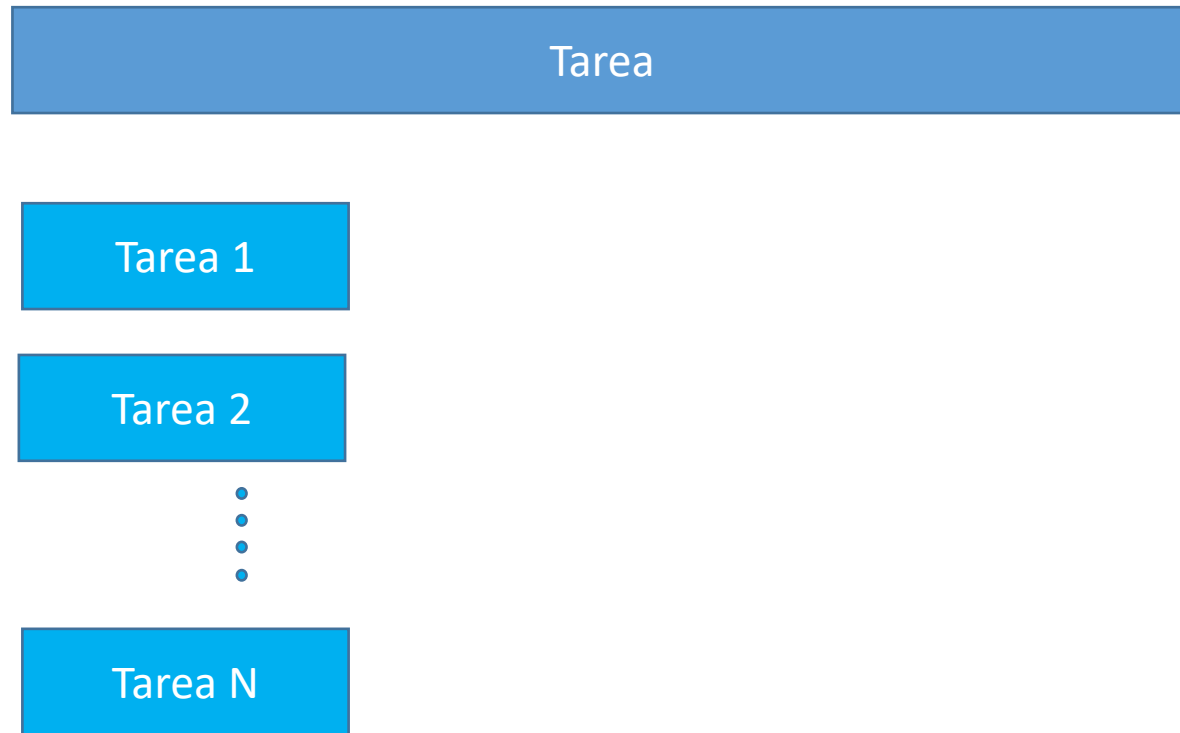
- Dividir la tarea en múltiples tareas.
- Creación de hilos, pasando tareas a hilos.
- Tiempo entre `thread.start()` para programar el hilo.
- Tiempo hasta que finaliza el último hilo y señala.
- Tiempo hasta que se ejecuta el subproceso de agregación.
- Agregación de los subresultados en un solo artefacto.

Costo inherente de paralelización y agregación

Latency vs Original Task Latency



Tipo 1 Paralelización de Tareas



Tipo 2: tarea secuencial irrompible



Un Hilo, secuencial

Tipo 3 - Parcialmente Paralelizable, Parcialmente Secuencial

